

1. Variadic Function – Weighted Average

Concept:

A variadic function is a function that can accept a variable number of arguments.

Algorithm:

1. Start.
2. Define the function `weightedAverage(int count, ...)`.
3. Check whether count is positive.
4. Initialize a variable argument list using `va_start()`.
5. Read each value and its corresponding weight using `va_arg()`.
6. Check whether the weight is negative.
7. Calculate:
 - sum of value × weight
 - total of all weights.
8. End the variable argument processing using `va_end()`.
9. If total weight is zero, display an error.
10. Otherwise, calculate and return the weighted average.
11. Display the result in `main()`.
12. Stop.

C Program:

```
#include <stdio.h>
```

```
#include <stdarg.h>
```

```
double weightedAverage(int count, ...)
```

```
{
```

```
    va_list args;
```

```
    double value, weight;
```

```
    double weightedSum = 0.0;
```

```
double totalWeight = 0.0;

int i;

if (count <= 0)
{
    printf("Error: Count must be positive.\n");
    return -1.0;
}

/* Start accessing variable arguments */
va_start(args, count);

for (i = 0; i < count; i++)
{
    /* Get value and weight */
    value = va_arg(args, double);
    weight = va_arg(args, double);

    /* Check for negative weight */
    if (weight < 0)
    {
        printf("Error: Weight cannot be negative.\n");
        va_end(args);
        return -1.0;
    }

    weightedSum = weightedSum + (value * weight);
    totalWeight = totalWeight + weight;
}
```

```

}

/* End accessing variable arguments */
va_end(args);

/* Check if total weight is zero */
if (totalWeight == 0)
{
    printf("Error: Total weight cannot be zero.\n");
    return -1.0;
}

return weightedSum / totalWeight;
}

int main()
{
    double result;

    result = weightedAverage(3,
        80.0, 0.3,
        75.0, 0.2,
        90.0, 0.5);

    if (result != -1.0)
    {
        printf("Weighted Average = %.2f\n", result);
    }
}

```

```
    return 0;
}
```

2. Command-Line Arguments – Integer Operations

In this program, the input is not entered using scanf().

Algorithm:

1. Start.
2. Accept values through command-line arguments.
3. Check every argument to determine whether it is a valid integer.
4. Display an error message for each invalid argument.
5. Store valid integers in an array.
6. Sort the valid integers in ascending order.
7. Display the sorted integers.
8. Find and display:
 - Minimum
 - Maximum
 - Average
 - Median
 - Second-largest distinct value
9. Stop.

C Program:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <ctype.h>
```

```
/* Function declarations */
```

```
int isInteger(char str[]);
```

```
void sortArray(int arr[], int n);  
double findAverage(int arr[], int n);  
double findMedian(int arr[], int n);  
int findSecondLargest(int arr[], int n, int *found);
```

```
int main(int argc, char *argv[])  
{  
    int arr[100];  
    int n = 0;  
    int i;  
    int secondLargest;  
    int found;  
  
    /* Check whether arguments are provided */  
    if (argc == 1)  
    {  
        printf("No integers provided.\n");  
        return 0;  
    }  
  
    /* Validate command-line arguments */  
    for (i = 1; i < argc; i++)  
    {  
        if (isInteger(argv[i]))  
        {  
            arr[n] = atoi(argv[i]);  
            n++;  
        }  
    }
```

```
    else
    {
        printf("Invalid argument ignored: %s\n", argv[i]);
    }
}
```

```
/* Check whether any valid integers exist */
```

```
if (n == 0)
{
    printf("No valid integers found.\n");
    return 0;
}
```

```
/* Sort the array */
```

```
sortArray(arr, n);
```

```
printf("\nValid integers in ascending order: ");
```

```
for (i = 0; i < n; i++)
{
    printf("%d ", arr[i]);
}
```

```
printf("\n");
```

```
/* Minimum and maximum after sorting */
```

```
printf("Minimum: %d\n", arr[0]);
```

```
printf("Maximum: %d\n", arr[n - 1]);
```

```
printf("Average: %.2f\n", findAverage(arr, n));
printf("Median: %.2f\n", findMedian(arr, n));

/* Find second-largest distinct value */
secondLargest = findSecondLargest(arr, n, &found);

if (found)
{
    printf("Second-largest distinct value: %d\n",
        secondLargest);
}
else
{
    printf("Second-largest distinct value does not exist.\n");
}

return 0;
}
```

```
/* Check whether a string represents an integer */
int isInteger(char str[])
{
    int i = 0;

    /* Allow + or - sign */
    if (str[0] == '+' || str[0] == '-')

```

```
{  
    i = 1;  
}  
  
/* Sign alone is not an integer */  
if (str[i] == '\0')  
{  
    return 0;  
}  
  
while (str[i] != '\0')  
{  
    if (!isdigit(str[i]))  
    {  
        return 0;  
    }  
  
    i++;  
}  
  
return 1;  
}
```

```
/* Sort array using Bubble Sort */  
void sortArray(int arr[], int n)  
{  
    int i, j, temp;
```



```
for (i = 0; i < n - 1; i++)
{
    for (j = 0; j < n - i - 1; j++)
    {
        if (arr[j] > arr[j + 1])
        {
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
}
```

```
/* Calculate average */
double findAverage(int arr[], int n)
{
    int i;
    double sum = 0;

    for (i = 0; i < n; i++)
    {
        sum = sum + arr[i];
    }

    return sum / n;
```

```
}
```

```
/* Calculate median */
```

```
double findMedian(int arr[], int n)
```

```
{
```

```
    if (n % 2 == 0)
```

```
    {
```

```
        return (arr[n / 2 - 1] + arr[n / 2]) / 2.0;
```

```
    }
```

```
    else
```

```
    {
```

```
        return arr[n / 2];
```

```
    }
```

```
}
```

```
/* Find second-largest distinct value */
```

```
int findSecondLargest(int arr[], int n, int *found)
```

```
{
```

```
    int i;
```

```
    int largest = arr[n - 1];
```

```
    *found = 0;
```

```
    for (i = n - 2; i >= 0; i--)
```

```
    {
```

```
        if (arr[i] != largest)
```

```
    {  
        *found = 1;  
        return arr[i];  
    }  
}  
  
return 0;  
}
```